

Smart Pointers & Memory Interview Questions

Բոլոր հնարավոր հարցերը (Memory leak, RAII, new/delete, unique/shared/weak_ptr)

Այս ֆայլը memory management-ի interview հարիւրի ambogjakan zhoxovacu e. Erek mas.

Մաս A — 35 multiple-choice հարց. Մաս B — 30 bac (open) հարց. Մաս C — 15 tricky kod / complexity հարց.

Bolor patasxannery **miayn verji ejerum** en (Answer Key A, B, C).

ՄԱՍ A — Multiple Choice (35)

Q1

Memory leak inch e? - (a) shat memory ogtagorcel - (b) heap memory vercvac bayc chi azatvel u hascee korel - (c) stack overflow - (d) null pointer

Q2

Vory-n e memory leak-i patchar? - (a) delete mo'racum - (b) early return aranc delete - (c) pointer overwrite - (d) bolory

Q3

Memory leak haytnaberelu tool? - (a) valgrind - (b) AddressSanitizer - (c) erku-n el - (d) ochmeky

Q4

RAII inch e nshanakum? - (a) Random Access... - (b) Resource Acquisition Is Initialization - (c) Rapid Allocation... - (d) Reference And Index Init

Q5

RAII-um resource-y erb e azatvum? - (a) constructor-um - (b) destructor-um (scope verj) - (c) main-i verjum - (d) dzerqov

Q6

RAII-i himnakan aʻravelutyuny? - (a) arag - (b) exception-safe cleanup, chka leak - (c) qich kod - (d) sorted

Q7

new vs malloc himnakan tarberutyun? - (a) ochmi - (b) new-y ctor/dtor kanchum, malloc-y voych - (c) malloc-y aveli arag - (d) new-y C-i

Q8

new anhajoxutyan depqum inch e anum? - (a) return nullptr - (b) throw std::bad_alloc - (c) crash - (d) 0

Q9

malloc anhajoxutyan depqum? - (a) throw - (b) return nullptr - (c) crash - (d) -1

Q10

Kareli e new-i het free oqtagorcel? - (a) ayo - (b) voych (undefined behavior) - (c) miayn int-i hamar - (d) misht

Q11

new[]-i zuygy inch e? - (a) delete - (b) delete[] - (c) free - (d) free[]

Q12

unique_ptr-i ownership model? - (a) shared - (b) exclusive (mek owner) - (c) non-owning - (d) weak

Q13

unique_ptr copy linum e? - (a) ayo - (b) voych (miayn move) - (c) miayn int-i - (d) misht

Q14

unique_ptr ownership inchpes es poxancum? - (a) copy - (b) std::move - (c) get() - (d) reset()

Q15

Naxentreli unique_ptr steghcum? - (a) unique_ptr(new int) - (b) make_unique() - (c) malloc - (d) new

Q16

shared_ptr-i ownership model? - (a) exclusive - (b) shared (mi qani owner) - (c) non-owning - (d) ochmi

Q17

shared_ptr-y erb e resource-y delete anum? - (a) araji owner ochncelis - (b) verjin owner ochncelis (count 0) - (c) erbek - (d) main verj

Q18

use_count() inch e talis? - (a) memory chap - (b) qani strong owner - (c) qani weak - (d) bucket

Q19

Naxentreli shared_ptr steghcum? - (a) shared_ptr(new int) - (b) make_shared() - (c) malloc - (d) new

Q20

make_shared-i aravelutyuny? - (a) 2 allocation - (b) 1 allocation (resource+control block) - (c) aveli dandaghn - (d) ochmi

Q21

Control block inch e uqucum? - (a) miayn tvyal - (b) strong count + weak count + deleter - (c) miayn count - (d) bucket

Q22

shared_ptr-i amenamec problem? - (a) arag - (b) circular reference (leak) - (c) copy chka - (d) null

Q23

Circular reference-i lucum? - (a) unique_ptr - (b) weak_ptr - (c) raw pointer - (d) malloc

Q24

weak_ptr achecnum e strong count-y? - (a) ayo - (b) voch - (c) miayn copy - (d) misht

Q25

weak_ptr-ic resource-i anvtang mutq? - (a) *wp - (b) wp.lock() - (c) wp.get() - (d) wp->

Q26

expired() inch e talis? - (a) memory chap - (b) resource jnkvac e? (true/false) - (c) count - (d) bucket

Q27

Default smart pointer entrutyun? - (a) shared_ptr - (b) unique_ptr - (c) weak_ptr - (d) raw

Q28

Rule of Zero inch e? - (a) 0 member - (b) RAII tiper -> ochmi dzerqov dtor/copy/move - (c) 0 constructor - (d) 0 allocation

Q29

Rule of Three-i erek-y? - (a) 3 ctor - (b) dtor, copy-ctor, copy-assign - (c) 3 pointer - (d) 3 member

Q30

Rule of Five-y avelacnum e? - (a) 5 ctor - (b) move-ctor + move-assign - (c) 5 member - (d) ochmi

Q31

Dangling pointer inch e? - (a) null pointer - (b) jnkvac memory-i cuyc tvox pointer - (c) mec pointer - (d) shared

Q32

Double delete inch e? - (a) 2 new - (b) nuyn pointer 2 angam delete (UB) - (c) delete[] - (d) free

Q33

unique_ptr-i overhead? - (a) mec - (b) grethe zero (raw-i chapov) - (c) 2x - (d) atomic

Q34

shared_ptr reference count-y inch e? - (a) sovorakan int - (b) atomic (thread-safe) - (c) float - (d) chka

Q35

release() vs reset() unique_ptr-um? - (a) nuyny - (b) release ownership hanum (delete chi); reset azatum (delete) - (c) release delete anum - (d) reset ownership hanum

ՄԱՍ B — Open Questions (30)

1. Memory leak inch e, inchpes e linum, inchpes xusapel.
2. Dangling pointer vs double delete.
3. Memory leak inchpes es haytnaberum (tools).
4. RAII inch e, inchpes e ashxatum.
5. RAII inchpes e ashxatum exception-i depqum (stack unwinding).
6. RAII real orinakner (memory, file, lock).
7. new vs malloc - amboxjakan tarberutyun.
8. new anhajoxutyan depqum inch (bad_alloc).
9. Inchu new-y aveli lav e class-eri hamar (constructor).
10. delete vs delete[].
11. Inchu chi kareli new+free xaʔnel.
12. unique_ptr inch e, exclusive ownership.
13. Inchu unique_ptr copy chka.
14. make_unique inchu naxentrel.
15. unique_ptr ownership transfer (move).
16. get() vs release() vs reset().
17. shared_ptr inch e, shared ownership.
18. Reference counting inchpes e ashxatum.
19. Control block inch e.
20. make_shared vs shared_ptr(new).
21. Circular reference problem-y.
22. weak_ptr inch e, inchu e petq.
23. weak_ptr-ov circular reference lucum.
24. lock() inch e anum, inchu petq.
25. expired() inch e.
26. Vory smart pointer erb ogtagorcel.
27. Rule of Zero / Three / Five.

- 28. shared_ptr overhead - inchi.
 - 29. Smart pointers inchi en petq (raw problems).
 - 30. Rule of Zero inchi e lavaguyny.
-

UUU C — Tricky Code (15)

C1

```
void f() {  
    int* p = new int(5);  
    return;  
}
```

- (a) OK
- (b) memory leak (delete chka)
- (c) CE
- (d) crash

C2

```
std::unique_ptr<int> a = std::make_unique<int>(5);  
std::unique_ptr<int> b = a;
```

- (a) OK
- (b) compile error (unique copy chka)
- (c) runtime error
- (d) leak

C3

```
std::unique_ptr<int> a = std::make_unique<int>(5);  
std::unique_ptr<int> b = std::move(a);  
cout << (a == nullptr);
```

- (a) 0
- (b) 1 (a darav nullptr)
- (c) CE

- (d) UB

C4

```
auto p = std::make_shared<int>(10);  
auto q = p;  
cout << p.use_count();
```

- (a) 1
- (b) 2
- (c) 0
- (d) CE

C5

```
int* p = new int(5);  
delete p;  
delete p;
```

- (a) OK
- (b) undefined behavior (double delete)
- (c) CE
- (d) leak

C6

```
int* p = new int(5);  
free(p);
```

- (a) OK
- (b) undefined behavior (new+free)
- (c) CE
- (d) leak

C7

```
int* arr = new int[10];  
delete arr;
```

- (a) OK

- (b) undefined behavior (delete vs delete[])
- (c) CE
- (d) leak

C8

```
struct Node { std::shared_ptr<Node> next; };
auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b; b->next = a;
```

- (a) OK, azatvum
- (b) circular reference -> leak
- (c) CE
- (d) crash

C9

```
auto sp = std::make_shared<int>(10);
std::weak_ptr<int> wp = sp;
cout << sp.use_count();
```

- (a) 1 (weak chi achecnum)
- (b) 2
- (c) 0
- (d) CE

C10

```
auto sp = std::make_shared<int>(10);
std::weak_ptr<int> wp = sp;
sp.reset();
cout << wp.expired();
```

- (a) 0
- (b) 1 (resource jnjvac)
- (c) CE
- (d) UB

C11

```
std::weak_ptr<int> wp;
{
    auto sp = std::make_shared<int>(5);
    wp = sp;
}
if (auto s = wp.lock()) cout << "alive";
else cout << "gone";
```

- (a) alive
- (b) gone (sp scope-ic durs, jn jvac)
- (c) CE
- (d) UB

C12

```
class A { int* p; public: A(){p=new int[10];} ~A(){delete[] p;} };
A a1;
A a2 = a1;
```

- (a) OK
- (b) shallow copy -> double delete (UB) [Rule of Three xaxtum]
- (c) CE
- (d) leak

C13

```
A* p = (A*)malloc(sizeof(A)); // A uni constructor
```

- (a) constructor kanchvum
- (b) constructor CHI kanchvum (raw memory)
- (c) CE
- (d) crash

C14

```
auto p = std::make_unique<int>(5);  
int* raw = p.release();  
cout << (p == nullptr);
```

- (a) 0
- (b) 1 (release ownership hanec) + raw petq e delete
- (c) CE
- (d) UB

C15

```
{  
    auto a = std::make_shared<int>(1);  
    {  
        auto b = a;  
        cout << a.use_count() << " ";    // ?  
    }  
    cout << a.use_count();                // ?  
}
```

- (a) 2 1
- (b) 1 1
- (c) 2 2
- (d) 1 2

ANSWER KEY A

1-b · 2-d · 3-c · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b 11-b · 12-b · 13-b · 14-b · 15-b · 16-b · 17-b · 18-b ·
19-b · 20-b 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-b · 28-b · 29-b · 30-b 31-b · 32-b · 33-b · 34-b · 35-
b

ANSWER KEY B (hakirch)

1. Heap memory vercvac bayc chi azatvel, hascee korel. Patchar `delete moracum, early return/exception, overwrite`. Xusapel RAII/smart pointers.
2. Dangling `jnjvac memory-i cuyc tvox pointer (use-after-free)`. Double delete nuyn pointer 2 angam delete. Erku-n el UB.
3. Valgrind (`--leak-check=full`), AddressSanitizer/LeakSanitizer (`-fsanitize=address/leak`), CRT debug heap.
4. Resource Acquisition Is Initialization. resource vercvum ctor-um, azatvum dtor-um; scope verj -> avtomat cleanup.
5. Exception-i zham stack unwinding -> C++-y avtomat kanchum local object-neri destructor-nery -> cleanup. Dra hamar RAII exception-safe e.
6. `unique_ptr/shared_ptr` (memory), `lock_guard` (mutex), `fstream` (file), `vector/string`.
7. new `ctor/dtor kanchum, type-safe, throw bad_alloc, operator. malloc` voch ctor, void* return, nullptr, function, uni realloc.
8. `throw std::bad_alloc` (malloc-y nullptr).
9. Constructor kanchum -> object chisht init; malloc-y miayn raw memory (uninitialized).
10. `new->delete`, `new[]->delete[]`. Sxal zuyg -> UB.
11. Tarber allocator/mechanism (new operator vs malloc function); xaŋnely -> UB.
12. Exclusive ownership -- mek resource mek owner; scope verj avtomat delete.
13. Exclusive ownership -- copy -> erku owner -> double delete. Miayn move.
14. Aveli karch, exception-safe, chka baciahayt new.
15. `std::move` -> hin pointer nullptr.
16. `get()` raw (ownership pahum); `release()` ownership hanum raw talis (delete chi); `reset()` azatum (delete).
17. Shared ownership -- mi qani owner; verjin owner ochncelis delete (ref count).
18. `copy count++`, `ochncum count--`, count 0 delete.
19. Strong count + weak count + deleter; bolor copy nuyn control block.
20. `make_shared` 1 allocation (arag, exception-safe); (new) 2.
21. Erku `shared_ptr` irar cuyc talis -> count erbek 0 -> leak.
22. Non-owning smart pointer, chi achecnum count; circular lucum, observer/cache.
23. `weak_ptr` mek uxxutyamb (parent->child shared, child->parent weak) -> cycle kotrvum.
24. Veradarcnum `shared_ptr` (ete kar e) kam nullptr; anvtang mutq.

- 25. true ete resource jnjbac (strong count 0).
 - 26. unique default; shared iskapas shared petq e; weak circular/observer.
 - 27. Three (dtor/copy-ctor/copy-assign), Five (+move), Zero (RAII -> ochmi).
 - 28. Atomic ref count + control block -> unique-ic canr.
 - 29. Raw problem-neri (leak/dangling/double-delete/ownership) lucum RAII-ov.
 - 30. Ochmi dzerqov special member -> qich kod, qich sxal, RAII tiper.
-

ANSWER KEY C

C1-b · C2-b · C3-b · C4-b · C5-b · C6-b · C7-b · C8-b · C9-a · C10-b C11-b · C12-b · C13-b · C14-b · C15-a

Bacatrutyunner (C)

C1 new ařanc delete -> leak. **C2** unique copy chka -> CE. **C3** move heto a -> nullptr -> 1. **C4** copy -> use_count 2. **C5** double delete -> UB. **C6** new+free -> UB. **C7** new[] + delete (ne delete[]) -> UB. **C8** circular shared_ptr -> leak. **C9** weak chi achecnum -> use_count 1. **C10** reset -> resource jnjbac -> expired true (1). **C11** sp scope-ic durs -> resource jnjbac -> lock() nullptr -> "gone". **C12** Rule of Three xaxtum -> shallow copy -> double delete UB. **C13** malloc -> constructor chi kanchvum. **C14** release -> p nullptr (1), raw-y petq e dzerqov delete. **C15** inner scope count 2, durs galuc 1 -> "2 1".

Endhanur kanonner

```
LEAK: new ařanc delete | RAII/smart pointer -> xusapel
new->delete | new[]->delete[] | malloc->free | MI XAŘNIR (UB)
unique: copy CHKA (move) | shared: ref count, circular->leak | weak: lock()/expired()
Rule: Zero (RAII) > Five > Three
make_unique / make_shared -> NAXENTRELI
```